

PROPUESTA DE

PROYECTO DE INVESTIGACIÓN

Informe del Primer Parcial · Planificación y Diseño



NEXUSFORGE OS

PLATAFORMA COLABORATIVA Y GAMIFICADA PARA
PROYECTOS DE INGENIERÍA

Universidad Tecnológica de Honduras (UTH)

Asignatura: Programación para Sistemas Abiertos II

Catedrático: Ing. Luis Henríquez Valle Yanes

Presentado por: **Ana Leticia Montes Sarmiento**

Cuenta: 202120030068

JUNIO 2026

Índice

Introducción	3
Objetivos	3
1. Estudio inicial	3
1.1 Problema y estado del arte	3
1.2 Mercado, nicho y usuarios	4
1.3 Benchmarking	4
1.4 Estrategia de monetización	4
1.5 Viabilidad técnica, económica y estratégica	5
2. Definición de requisitos	6
2.1 Roles y permisos	6
2.2 Funcionalidades esenciales y opcionales	6
2.3 Alcance del proyecto	6
3. Diseño de la solución	8
3.1 Arquitectura del sistema y stack	8
3.2 Modelo de datos	8
3.3 Experiencia de usuario (UI/UX)	8
3.4 Riesgos y mitigaciones	9
Conclusión	10

Introducción

El presente informe corresponde al **Primer Parcial** de la asignatura Programación para Sistemas Abiertos II y reúne el trabajo de **planificación y diseño** de mi proyecto: **NexusForge OS**, una plataforma web colaborativa, gamificada y en tiempo real para la gestión de proyectos de ingeniería de software académicos, desplegada sobre un servidor Linux.

La idea consiste en consolidar en un solo ecosistema la gestión de entregables, el seguimiento de tareas mediante un tablero Kanban, la comunicación en vivo del equipo y la presentación pública de los proyectos en una dinámica competitiva de votos y rangos. En las siguientes páginas documento el estudio inicial, la definición de requisitos y el diseño de la solución, sin escribir todavía código, tal como lo exige esta primera etapa.

Objetivos

Objetivo general

Diseñar una plataforma web colaborativa y gamificada que facilite la gestión, el seguimiento y la presentación de proyectos académicos de ingeniería de software mediante herramientas de colaboración en tiempo real.

Objetivos específicos

- Centralizar la gestión de los proyectos académicos en un solo ecosistema.
- Facilitar la comunicación entre estudiantes y docentes.
- Implementar mecanismos de gamificación para aumentar la participación.
- Permitir el seguimiento visual del avance mediante tableros Kanban.

1. Estudio inicial

1.1 Problema y estado del arte

Al trabajar en proyectos de clase he identificado tres problemas recurrentes que NexusForge OS busca resolver:

- **Dispersión de entregables:** el código, la documentación y los videos demostrativos quedan repartidos en herramientas distintas.
- **Comunicación poco contextual:** la coordinación ocurre en canales externos no estructurados, separados del proyecto.
- **Poca visibilidad del avance grupal:** no hay forma clara de ver quién tiene asignada cada tarea ni en qué estado va.

Revisé las soluciones existentes y encontré que los gestores tradicionales (Google Classroom, Trello, GitHub Classroom) cubren piezas sueltas pero permanecen fragmentados y poco motivadores. Por otro lado, plataformas de alta densidad de información como **Discord** y **Twitch** logran experiencias muy participativas. Mi propuesta toma lo mejor de ambos mundos: la gestión académica de las primeras y la dinámica social y en vivo de las segundas.

1.2 Mercado, nicho y usuarios

El nicho al que apunto es la **comunidad de ingeniería de software de la universidad**: estudiantes que desarrollan proyectos a lo largo del periodo y docentes que los supervisan y evalúan. El sistema contempla tres perfiles de usuario claramente diferenciados:

Usuario	Rol en la plataforma
Maestro	Crea las clases y sus códigos de acceso, supervisa los grupos y evalúa los proyectos.
Estudiante	Forma grupos, gestiona el tablero Kanban, vincula su repositorio, chatea en vivo y publica su proyecto.
Visitante	Explora los proyectos públicos y los califica; su interacción es secundaria y no altera los rangos oficiales.

1.3 Benchmarking

Comparé mi propuesta con las herramientas que hoy usamos por separado. Ninguna reúne, en un solo lugar, la gestión académica, la colaboración en tiempo real y la presentación gamificada de los proyectos:

Plataforma	Tablero de tareas	Evaluación docente	Tiempo real	Gamificación
Google Classroom	Parcial	Sí	No	No
Trello / Jira	Sí	No	Parcial	No
GitHub Classroom	Parcial	Parcial	No	No
NexusForge OS	Sí	Sí	Sí	Sí

1.4 Estrategia de monetización

Aunque su primera versión es académica, la plataforma admite varias vías de ingreso a futuro:

- **Licencia institucional (B2B)**: las universidades pagan por usar la plataforma con sus clases.
- **Plan premium para estudiantes**: más almacenamiento y estadísticas avanzadas de su proyecto.
- **Vinculación con empresas**: acceso de reclutadores a los proyectos mejor calificados.

- **Publicidad segmentada** orientada al sector tecnológico y educativo.

1.5 Viabilidad técnica, económica y estratégica

Técnica: el proyecto se apoya en tecnologías abiertas y maduras (Next.js, PostgreSQL, Node.js, Socket.IO) sobre servidor Linux, con Supabase para autenticación y almacenamiento, lo que reduce el riesgo de implementación. **Económica:** el costo es bajo, pues todo el núcleo es de uso gratuito u open source y puede iniciarse con planes sin costo. **Estratégica:** atiende una necesidad real de la comunidad de ingeniería y su enfoque gamificado constituye un diferenciador claro frente a las herramientas actuales.

El siguiente estimado de costos refuerza la viabilidad económica del proyecto:

Recurso	Costo aproximado
Dominio	\$10 – \$15 / año
VPS Linux	\$5 – \$10 / mes
Supabase	Plan gratuito inicial
Desarrollo	Realizado por la estudiante (sin costo)

2. Definición de requisitos

2.1 Roles y permisos

Para proteger la integridad de las evaluaciones, el sistema funciona con un **control de acceso basado en roles (RBAC)**. La siguiente matriz resume las acciones permitidas a cada perfil:

Acción	Maestro	Estudiante	Visitante
Crear clases y generar códigos de acceso	Sí	No	No
Crear grupos e invitar miembros	Sí	Sí	No
Vincular repositorio de GitHub y URL de despliegue	No	Sí	No
Gestionar el tablero Kanban del grupo	No	Sí	No
Enviar mensajes en el chat en vivo	No	Sí	No
Ver proyectos públicos y calificar con estrellas	Sí	Sí	Sí
Aportar monedas válidas en los duelos	Sí	Sí	No

2.2 Funcionalidades esenciales y opcionales

Separé las funciones en un núcleo imprescindible y un conjunto opcional que aporta la identidad gamificada:

Funcionalidades esenciales	Funcionalidades opcionales
• Registro e inicio de sesión con los tres roles. • Gestión de clases y grupos con códigos de acceso. • Tablero Kanban de tareas (TODO, DOING, DONE). • Perfil de proyecto con repositorio, demo y despliegue. • Chat en vivo del grupo (tiempo real). • Base de datos PostgreSQL conectada.	• Duelos públicos de proyectos. • Economía de votos y monedas. • Calificación con estrellas y comentarios. • Rangos automáticos (Bronce → Diamante). • Galería pública con bordes iluminados por rango. • Notificaciones en tiempo real.

2.3 Alcance del proyecto

El proyecto es una plataforma web responsiva sobre servidor Linux. Para mantener un alcance realista lo diseñé por capas: primero el núcleo funcional y, sobre él, las funciones gamificadas. El siguiente cuadro delimita el alcance:

Dentro del alcance	Fuera del alcance (por ahora)
• Gestión de clases, grupos y proyectos con los tres roles. • Tablero Kanban, chat en tiempo real y perfil del proyecto. • Calificaciones, votos y rangos de la comunidad. • Plataforma web responsiva sobre servidor Linux.	• Aplicación móvil nativa. • Pasarela de pagos real. • Integración con sistemas internos de cada universidad. • Funciones avanzadas de inteligencia artificial.

3. Diseño de la solución

3.1 Arquitectura del sistema y stack

Diseñé una arquitectura basada en un ecosistema homogéneo de **TypeScript**, que unifica los contratos de datos entre cliente y servidor. El stack seleccionado es:

Capa	Tecnología
Frontend	Next.js 14 (App Router) + Tailwind CSS
Tiempo real	Node.js + Socket.IO (servidor independiente)
Base de datos	PostgreSQL gestionada con Prisma ORM
Servicios integrados	Supabase (autenticación, base de datos y almacenamiento)
Servidor	Linux (contenedor dedicado para el tiempo real)

Durante el diseño detecté un punto crítico: los entornos *serverless* donde suele alojarse Next.js no mantienen conexiones WebSocket de larga duración. Por eso decidí **desacoplar el tiempo real** en un servidor Node.js + Socket.IO independiente y persistente, dejando a Next.js el renderizado y las llamadas REST. Además, para evitar fraudes en la votación, las interacciones de visitantes anónimos solo cuentan como popularidad informativa: las **monedas y los ascensos de rango requieren el voto de un usuario autenticado**.

3.2 Modelo de datos

Definé las entidades relacionales principales del sistema, pensadas para PostgreSQL con Prisma:

Entidad	Atributos principales
Usuario	id, nombre, email, rol (Maestro / Estudiante / Visitante)
Clase	id, nombre, accessCode, teacherId, createdAt
Grupo	id, nombre, isPublic, classId
Proyecto	id, título, githubRepoUrl, deployUrl, videoUrl, coins, ratingAvg, groupId
Tarea	id, título, descripción, dueDate, status (TODO/DOING/DONE), assignedToId, groupId
Comentario	id, contenido, createdAt, userId, projectId

3.3 Experiencia de usuario (UI/UX)

La interfaz se inspira en plataformas densas como Discord y Twitch, priorizando la inmersión técnica y evitando la fatiga visual. Definí estas directrices:

- **Tema Ultra-Dark:** fondos en grises oscuros y azul marino profundo, con texto en gris claro suave para largas sesiones de trabajo.
- **Acentos vivos:** violeta eléctrico y azul neón para estados activos, acciones y notificaciones en tiempo real.
- **Layout multi-panel:** panel izquierdo de navegación (clases y canales), panel central de trabajo (tablero, perfil del proyecto o comentarios) y panel derecho de presencia (miembros activos y chat).
- **Indicadores gamificados:** las tarjetas de proyecto muestran bordes iluminados según el rango alcanzado (por ejemplo, dorado para Oro y cian para Diamante).

3.4 Riesgos y mitigaciones

Identifiqué los principales riesgos del proyecto y definí una mitigación para cada uno:

Riesgo	Mitigación
Fraude en los votos	Autenticación obligatoria para votar.
Caída del servidor	Respaldos periódicos y monitoreo del servicio.
Pérdida de datos	Copias de seguridad automáticas de la base de datos.
Saturación de usuarios	Escalabilidad horizontal del servidor.

Conclusión

Con este primer parcial dejo planificada y diseñada una propuesta clara y realista. La investigación confirmó que hoy ninguna herramienta integra, en un mismo lugar, la gestión académica de proyectos, la colaboración en tiempo real y una dinámica gamificada de presentación. NexusForge OS nace para cubrir ese vacío en la comunidad de ingeniería de la universidad.

Definé los roles y permisos, las funcionalidades esenciales y opcionales, el alcance, la arquitectura, el modelo de datos y las directrices de experiencia de usuario. También anticipé dos riesgos técnicos —el tiempo real sobre entornos serverless y el fraude en la votación anónima— con sus respectivas mitigaciones. Todo ello me deja una base sólida y de bajo riesgo para las siguientes etapas del proyecto.